

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

ANSWERS:

1. Testing as an Engineering Activity in a Banking Application

Answer

Testing should be treated as a systematic engineering activity rather than an informal activity performed at the end of development. For the banking application, the testing process should begin with requirement analysis and continue through design, implementation, integration, system testing, and maintenance. Test objectives, risks, entry and exit criteria, test data, expected results, and responsibilities should be documented before execution.

Systematic test design can be introduced using techniques such as equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and risk-based testing. For example, for a fund-transfer limit of Rs. 1,00,000, boundary values such as Rs. 0, Rs. 1, Rs. 1,00,000 and Rs. 1,00,001 should be tested. Test management and defect tracking tools can be used to maintain test cases, execution results, traceability, and defects. Standards and organizational procedures should define how requirements are reviewed, test cases are prepared, defects are reported, and releases are approved.

These practices improve defect detection because testing becomes repeatable, measurable, and traceable. Ad-hoc testing can therefore be replaced by structured test cases, exploratory testing with defined charters, regression suites, automation for repetitive checks, and risk-based prioritization. For example, if customers previously reported duplicate transactions after repeated clicking on the transfer button, a systematic test would check rapid repeated clicks, network interruption, timeout, retry, and database consistency. Thus, engineering-based testing improves reliability, reduces escaped defects, and increases confidence in banking transactions.

2. Structured Testing Process for ATM Withdrawal

Answer

A structured testing process for an ATM should cover the complete Software Development Life Cycle (SDLC). During requirement analysis, testers review requirements such as card validation, PIN verification, balance checking, withdrawal limits, cash availability, receipt generation, and account updating. During test planning, scope, risks, resources, test data, environments, and entry and exit criteria are defined. During test design, positive, negative, boundary, security, usability, and recovery test cases are prepared.

During implementation and integration, unit-level and interface tests verify individual modules and their interactions. System testing then validates the complete withdrawal flow. For example, a normal test case may use a valid card, correct PIN, sufficient balance, and an amount within the daily limit. Negative cases should include an invalid PIN, insufficient balance, invalid amount, exceeded withdrawal limit, unavailable cash, blocked card, and communication failure. Boundary cases should test minimum and maximum permitted withdrawal amounts. Regression testing should be performed after changes to ensure that previously working functions remain correct.

The process should continue with defect logging, retesting of corrected defects, regression testing, and final reporting. Consistency is important because every release follows the same defined activities and quality criteria. For example, if a software change accidentally deducts money without dispensing cash, structured testing can detect the mismatch between account balance and cash output before deployment. Expected outcomes include fewer transaction errors, better traceability, consistent test coverage, improved security, and higher reliability of the ATM system.

3. Errors, Defects, and Failures in a Web Application

Answer

Error, defect, and failure are related but different testing concepts. An error is a human mistake made during requirements analysis, design, coding, or configuration. A defect is the fault introduced into the software as

a result of that error. A failure occurs when the executing software produces an incorrect or unexpected result that violates the expected behavior.

Consider a login operation. Suppose a developer misunderstands the requirement and writes code that accepts an incorrect password under a particular condition. The developer's misunderstanding is an error. The incorrect authentication logic present in the program is a defect. When a user enters an invalid password and the application nevertheless grants access, the observed incorrect behavior is a failure. Similarly, during a transaction, if the developer uses the wrong calculation logic, the resulting incorrect transaction amount is a failure caused by an underlying defect.

Confusing these terms affects testing because defect reports may become vague and developers may waste time reproducing or diagnosing the wrong problem. Clear reporting should identify the environment, preconditions, steps to reproduce, expected result, actual result, severity, priority, evidence, and affected module. For example, instead of reporting "login is wrong," a tester should state that after entering a valid username and invalid password three times, the system allows access, whereas the expected result is denial of access and account protection.

4. Defect Clustering and Test Case Prioritization

Answer

Defect clustering is the principle that a small number of modules often contain a large proportion of the defects. A testing team should use this observation when planning testing effort instead of distributing effort equally across all modules. Modules with high defect history, complex logic, frequent changes, security risks, or high business impact should receive greater testing priority.

For effective test planning, defect data from previous releases can be analyzed to identify high-risk modules. Test cases can then be prioritized according to risk, severity, business importance, and probability of failure. For example, in an online banking system, authentication, fund transfer, payment processing, and account balance modules should receive higher priority than rarely used informational pages. Boundary value analysis, decision tables, state transition testing, negative testing, and regression testing can be concentrated on these high-risk areas.

Defect clustering also supports efficient regression testing. If historical data shows that transaction processing produces most defects, the regression suite should execute transaction-related cases early in every release. A real-time example is an e-commerce application in which most defects are repeatedly found in the checkout and payment modules. The team can prioritize cart calculations, coupon rules, payment authorization, order confirmation, and refund scenarios before lower-risk pages.

Applying these principles improves testing efficiency because limited time and resources are directed toward areas where failures are most likely and most damaging. It also increases the probability of detecting critical defects before release, thereby improving software reliability and reducing the cost of late defect correction.

5. Defect Classification and Defect Repository

Answer

A defect repository is a structured collection of information about defects discovered during testing and operation. A company that maintains a repository but does not use it effectively may repeatedly introduce the same problems because historical knowledge is not converted into preventive test activities. The first step is to classify defects consistently according to their origin, type, severity, priority, affected module, and phase in which they were introduced or detected.

Defects may originate from incorrect requirements, design mistakes, coding errors, interface problems, database issues, configuration problems, or inadequate test data. They can be categorized as functional, performance, security, usability, compatibility, data-related, or integration defects. Severity indicates the impact of the defect, while priority indicates how urgently it should be fixed. The repository should record a

unique defect ID, description, steps to reproduce, expected and actual results, environment, screenshots or logs when appropriate, severity, priority, status, root cause, and resolution.

Repository data can directly support test design. If previous releases show repeated defects in validation, new test cases should specifically target missing, invalid, boundary, and unexpected input. If repeated integration defects occur between two modules, additional interface and end-to-end tests should be included in regression testing.

To prevent recurrence, the team should perform root-cause analysis, update regression suites, conduct reviews, improve coding and requirement practices, and monitor recurring defect categories. Effective use of the repository therefore changes it from a passive record into a source of testing knowledge and continuous quality improvement.

6. Root Cause Analysis for Mobile Banking Fund Transfer Failures

Answer

Frequent crashes during fund transfer should be investigated using a systematic testing and root-cause analysis approach. First, the tester should reproduce the crash consistently and collect information such as device model, operating system, application version, network condition, account state, transaction amount, logs, and crash traces. The failure should then be isolated to determine whether it is caused by input validation, memory handling, network communication, concurrency, database operations, or incorrect error handling.

Testing should cover normal, invalid, boundary, and interrupted transaction conditions. For example, test cases should include insufficient balance, maximum transfer amount, zero or negative values, expired sessions, slow network, loss of connectivity during transfer, repeated tapping of the transfer button, server timeout, and simultaneous transactions. Integration testing should verify communication between the mobile application, authentication service, transaction service, and database. Recovery testing should ensure that a failed transaction does not result in an incorrect debit or duplicate transfer.

Systematic regression testing should be executed whenever the transfer module is modified. Automated tests can repeatedly execute important transaction scenarios, while exploratory testing can investigate unusual user behavior. Security testing should also verify authorization and protection against unauthorized transfers.

For example, if a crash occurs when network connectivity is lost after the debit request is sent, the root cause may be inadequate transaction-state handling. The solution could include proper timeout handling, transaction identifiers, idempotent requests, and clear recovery logic. Thus, systematic testing not only identifies the immediate failure but also helps prevent similar crashes in future releases through regression tests and root-cause-based improvements.

7. Testing and Debugging

Answer

Testing and debugging are related but distinct software quality activities. Testing is the process of evaluating software to identify failures and determine whether it satisfies specified requirements. Debugging is the process performed after a failure or defect has been identified to locate its cause, correct the code or configuration, and verify the correction. In simple terms, testing reveals that something is wrong, while debugging investigates why it is wrong and fixes it.

For example, if a payroll system displays an incorrect salary, a tester executes the relevant test case and reports the mismatch between expected and actual results. The developer then debugs the program by examining the calculation logic, input values, variables, and execution path. After the defect is corrected, the tester retests the failed case and performs regression testing to confirm that the correction has not introduced new problems.

Confusing the two processes can lead to ineffective defect resolution. If developers only debug without systematic testing, many failures may never be discovered. Similarly, if testers attempt to modify source code rather than provide clear evidence of failure, defect investigation becomes difficult. A structured approach should therefore define separate responsibilities while maintaining collaboration between testers and developers.

The process should be: execute planned tests, record failures clearly, log defects with reproducible evidence, assign the defect for debugging, identify the root cause, implement the fix, retest the failed scenario, and perform regression testing. This integration improves software quality because testing provides independent evaluation and debugging provides corrective action. Together they reduce both existing defects and the chance of recurring failures.

8. Cross-Browser Testing for an E-Commerce Platform

Answer

Cross-browser testing is necessary because the same web application may behave differently across browsers, browser versions, operating systems, screen sizes, rendering engines, and device types. An e-commerce platform should first identify the browsers and versions most commonly used by its customers and then create a prioritized compatibility matrix.

Testing should cover critical workflows such as login, product search, product filtering, cart operations, checkout, payment, order confirmation, and account management. Functional tests should verify that each workflow produces the same expected result across supported browsers. UI and responsive testing should check layout, fonts, buttons, menus, images, forms, and mobile behavior. Compatibility defects may originate from unsupported CSS or JavaScript features, browser-specific rendering differences, outdated libraries, inconsistent event handling, or incorrect responsive rules.

Systematic testing can combine manual testing with automation. Browser automation tools can execute common regression scenarios across multiple browser environments, while cloud-based browser platforms can provide access to different operating systems and versions. Developer tools can also help identify console errors, network failures, and rendering problems.

For example, if the checkout button works in one browser but does not respond in another, the team should reproduce the issue, inspect browser-specific console errors and event handling, fix the underlying compatibility problem, and add the scenario to the regression suite. A risk-based browser matrix, automated regression tests, responsive testing, and regular compatibility checks provide better coverage. This approach improves reliability and gives customers a consistent shopping experience regardless of their supported browser.

9. Classification of Payroll Logic Defects

Answer

Incorrect salary calculations caused by logical errors should be classified carefully because defect classification helps determine the appropriate testing strategy. The defect may be categorized as a functional or calculation defect because the system does not implement the required payroll rules correctly. Depending on its origin, it may also be classified as a coding or business-logic defect. Its severity and priority should be assessed according to the financial and operational impact.

Payroll calculations should be tested using a combination of normal, boundary, equivalence partitioning, decision table, and data-driven testing. For example, salary calculations may depend on basic pay, allowances, deductions, tax rules, overtime, attendance, and employee category. Test data should cover employees with no deductions, maximum deductions, different tax brackets, overtime conditions, absent days, and boundary salary values. Expected results should be calculated independently so that the application output can be compared accurately.

Understanding defect classes improves testing because the team can select techniques that match the likely failure mechanism. A business-rule defect may require decision tables and rule-based test cases, while a boundary defect may require boundary value analysis. Historical classification can also reveal whether calculation errors repeatedly originate from requirements, coding, or data.

To prevent future releases from containing similar defects, requirements and payroll rules should be reviewed, calculations should be unit tested, code reviews should be performed, and a strong regression suite should be maintained. Automated tests can verify large sets of salary combinations after every change. Therefore, classification converts defect information into targeted test design and helps improve the accuracy and stability of payroll processing.

10. Structured Test Case Design Instead of Random Testing

Answer

Random testing without a defined objective can miss important conditions and provides poor traceability. The team should redesign the process by beginning with requirements, identifying risks, defining test conditions, designing test cases, preparing test data, executing tests, recording results, and maintaining regression cases. Each important requirement should be traceable to one or more test cases.

Structured test case design should include a unique test case ID, objective, preconditions, input data, execution steps, expected result, actual result, status, and relevant requirement reference. Techniques such as equivalence partitioning, boundary value analysis, decision tables, state transition testing, use-case testing, and error guessing can improve coverage. Risk-based prioritization should ensure that critical functionality is tested first.

For example, consider a login function. Instead of randomly entering usernames and passwords, the tester can create cases for valid credentials, invalid username, invalid password, both invalid, empty fields, maximum field length, locked account, expired password, repeated failed attempts, and session timeout. This provides systematic coverage of normal, negative, and boundary conditions.

Random or exploratory testing can still be useful, but it should complement rather than replace structured testing. Automation can execute stable repetitive regression cases, while exploratory testing can investigate unexpected behavior. The redesigned approach improves defect detection because test conditions are derived from requirements and risks rather than chance. It also makes testing repeatable, measurable, and auditable, leading to greater confidence in software quality.

11. Data Inconsistency in a Healthcare Application

Answer

Data inconsistency across healthcare modules can occur when the same patient information is stored or updated differently in multiple parts of the system. The testing approach should focus on data integrity, interface consistency, transaction handling, synchronization, and database correctness. The tester should first identify which modules share information, such as registration, appointments, doctor records, billing, and medical records.

Defects may originate from incorrect database mappings, inconsistent validation rules, duplicate records, failed transactions, incorrect update logic, integration errors, or concurrency problems. Systematic testing should verify that a change made in one module is correctly reflected in every dependent module. Database validation can compare application data with stored records, while integration tests can verify data flow between services and modules.

For example, if a patient's date of birth is updated in the registration module but the old value remains in the billing or medical-record module, the test should identify the mismatch and trace the data flow. Test cases should cover creation, update, deletion, synchronization failure, duplicate records, partial transactions, and concurrent updates. Boundary and negative data should also be included.

Defects should be tracked using structured reports containing the affected patient record or test identifier, module, steps, expected result, actual result, environment, severity, and evidence while avoiding unnecessary exposure of sensitive patient information. A centralized defect repository and traceability matrix can help identify repeated data-integrity problems. These practices improve consistency, reduce duplicate defects, and increase confidence that healthcare information remains accurate across the complete system.

12. Using a Defect Repository to Prevent Repeated Release Defects

Answer

Repeated defects despite testing indicate that historical defect information is not being effectively converted into preventive actions. A defect repository should be used not only to record individual issues but also to analyze patterns, identify root causes, and improve future test design. Each defect should contain information such as module, defect type, severity, priority, cause, detection phase, status, and resolution.

Repository data can support test design by identifying areas with high defect density and recurring failure types. For example, if several releases contain defects related to input validation, the team should expand validation tests using equivalence classes, boundary values, invalid inputs, and unexpected characters. If integration defects repeatedly occur between two modules, additional interface and end-to-end tests should be added.

Defect patterns can be identified by grouping repository records according to module, release, defect category, root cause, and severity. Trends may reveal that a particular component consistently produces defects or that defects are being introduced during a specific development phase. Root-cause analysis can then determine whether the underlying problem is ambiguous requirements, insufficient reviews, coding practices, configuration errors, or inadequate testing.

Improvements should include mandatory defect fields, consistent classification, regular defect trend analysis, root-cause analysis, regression test updates, and review of recurring defects before each release. Preventive actions should be tracked to completion rather than simply closing individual defects. Thus, the repository becomes a feedback mechanism for continuous improvement and helps reduce defect recurrence, testing effort, and release risk.

13. Testing a Web-Based Learning System Under Peak Usage

Answer

A web-based learning system that fails during peak usage indicates gaps in performance, load, stress, or scalability testing. Test planning should identify peak user volumes, critical user journeys, response-time expectations, concurrent sessions, database activity, and resource constraints. Functional correctness alone is not sufficient because the system must also remain stable and responsive when many users access it simultaneously.

Performance testing should include load testing with expected peak traffic, stress testing beyond expected capacity, endurance testing for long-duration usage, and volume testing for large amounts of data. Important scenarios may include simultaneous logins, course searches, video or material access, quiz submissions, result generation, and database-intensive operations. Test coverage should include realistic combinations of users rather than only isolated requests.

For example, if a learning platform is designed for 5,000 concurrent students during an examination, testing should simulate the expected peak and higher loads. The team should measure response time, throughput, error rate, CPU and memory usage, database performance, and server behavior. If failures occur only beyond a particular load, the capacity limit can be identified and addressed.

Better test strategies improve reliability because performance risks are detected before real users experience them. Improvements may include performance baselines, realistic workload models, automated load tests, capacity planning, monitoring, database optimization, caching, and scalability testing after major changes.

Therefore, proper planning and coverage ensure that the application is tested not only for correct outputs but also for dependable behavior under realistic operating conditions.

14. Early Involvement of Testers in the SDLC

Answer

Testers should be involved from the early stages of the SDLC rather than waiting until coding is completed. Early involvement allows testers to review requirements, identify ambiguity, define acceptance criteria, assess risks, and design test conditions before implementation. This supports the principle that defects should be prevented or detected as early as possible because correcting them later is generally more costly.

During requirement analysis, testers can check whether requirements are complete, consistent, measurable, and testable. During design, they can identify interface and integration risks. During development, testers can prepare test data and automation while developers perform unit testing. During integration and system testing, testers execute planned tests and report failures. This continuous participation improves communication between developers, testers, analysts, and stakeholders.

For example, suppose an online banking requirement states that a transaction should be considered successful after payment processing. A tester involved early may ask what should happen if the payment succeeds but the confirmation response is lost. This question can lead to clear requirements for transaction status, retries, and duplicate prevention before coding begins.

Team collaboration should include requirement reviews, joint planning meetings, shared acceptance criteria, traceability, defect triage, and continuous feedback. Early tester involvement therefore reduces ambiguity, prevents avoidable defects, improves testability, and decreases late rework. It also helps the entire team develop a shared understanding of quality rather than treating testing as a separate final-stage activity.

15. Improving Test Coverage and Validation

Answer

Incomplete test coverage can cause a system to work correctly for common cases while failing for less common or unexpected conditions. The first step is to identify coverage gaps by mapping requirements, functions, conditions, user workflows, and risks to test cases. A traceability matrix can help determine whether every important requirement has appropriate validation.

Test case design should include positive and negative scenarios, boundary values, equivalence classes, decision combinations, state transitions, and realistic end-to-end workflows. For example, if an application accepts an age from 18 to 60, testing only ages such as 25 and 40 is insufficient. Values such as 17, 18, 19, 59, 60, and 61 should also be tested. If a feature has several conditions, decision table testing can ensure that important combinations are not omitted.

Coverage should not be considered only as the percentage of executed tests. The quality of the selected tests and the importance of the conditions covered are also important. Risk-based testing should prioritize security, financial, safety, and high-usage functions. Regression suites should be updated whenever new defects are discovered so that the same failure is not reintroduced.

Validation can be improved through requirement reviews, independent expected-result calculation, automated regression tests, exploratory testing, boundary analysis, and defect-based test design. For example, if a defect is found in a discount calculation, additional cases should be created for zero discount, minimum and maximum discount, invalid coupon codes, expired coupons, and combinations with taxes. Better coverage therefore increases reliability and reduces failures caused by untested scenarios.

16. Improving Defect Reporting in a Logistics Application

Answer

Poor defect reporting can delay identification, reproduction, prioritization, and correction of issues. A logistics application should use a standardized defect reporting process so that every reported problem contains enough information for the development team to understand and reproduce it. Clear definitions of error, defect, and failure also help testers describe problems accurately.

A structured defect report should include a unique identifier, concise title, module, environment, build version, preconditions, steps to reproduce, expected result, actual result, severity, priority, frequency, and supporting evidence such as logs or screenshots when appropriate. For example, instead of reporting “delivery tracking is not working,” a clearer report would state that after entering a valid tracking number and selecting Track, the system displays an old location while the expected result is the latest recorded location.

Defect reports should distinguish severity from priority. A critical failure that prevents shipment processing may have high severity and high priority, while a minor visual issue may have low severity even if it is scheduled for a later release. A common workflow can be used for statuses such as New, Assigned, In Progress, Fixed, Retest, Reopened, and Closed.

Reporting standards should be supported by templates, mandatory fields, reviewer checks, clear classification, and a centralized repository. Regular defect triage meetings can help teams agree on priority and ownership. These practices improve communication because testers, developers, and managers work from the same information. As a result, defects can be reproduced faster, fixed more accurately, and tracked throughout their lifecycle.

17. Process-Based Testing for Consistency Across Modules

Answer

A process-based testing approach can solve inconsistency by defining a common testing lifecycle that is followed across all software modules. The process should specify activities, responsibilities, inputs, outputs, templates, review points, and completion criteria. This ensures that testing does not depend on individual tester habits.

A standardized process may include requirement review, test planning, risk assessment, test design, peer review of test cases, test environment preparation, execution, defect reporting, retesting, regression testing, and test closure. Common test case formats and defect templates should be used across modules. A traceability matrix can connect requirements to test cases and results, while a shared repository can store reusable test assets.

For example, consider an e-commerce application containing login, product, cart, payment, and order modules. Each module should follow the same basic test design and defect reporting standards. However, risk-based priorities can differ: payment and order processing may receive more extensive testing than static product information. Standardization does not mean identical tests; it means applying a consistent quality process while adapting test depth to risk.

Consistency can be maintained through checklists, peer reviews, training, common tools, automation standards, regular audits, and measurement of coverage and defect trends. Continuous improvement should be based on lessons learned from previous releases. A process-based approach therefore reduces omissions, improves repeatability, makes results easier to compare, and increases overall software quality across modules.

18. Boundary Testing for Untested Edge Cases

Answer

Edge cases are often missed because testers concentrate on normal user behavior. Boundary value analysis is particularly useful because defects frequently occur at the limits of valid and invalid input ranges. When a

system fails after deployment due to an edge case, the test process should be reviewed to determine whether requirements, equivalence classes, boundaries, negative cases, or unusual state transitions were omitted.

For example, if a system accepts a transaction amount between Rs. 100 and Rs. 50,000, boundary tests should include Rs. 99, Rs. 100, Rs. 101, Rs. 49,999, Rs. 50,000, and Rs. 50,001. The same principle can be applied to character limits, dates, file sizes, account balances, quantities, and time periods. Invalid and extreme combinations should also be considered.

To include edge cases systematically, testers should derive test conditions directly from requirements and identify every input range and rule. Equivalence partitioning can reduce the number of representative cases, while boundary value analysis targets the most error-prone points. Decision tables can cover combinations of conditions, and error guessing can add cases based on experience and historical defects.

The test suite should be reviewed by peers and updated whenever a production defect is discovered. Regression testing should ensure that corrected boundary conditions remain stable in later releases. Therefore, systematic test design prevents edge cases from being treated as accidental discoveries and improves confidence that the software behaves correctly at both normal and extreme operating conditions.

19. Defect Documentation and Repository Management

Answer

Proper defect documentation is essential for tracking, analysis, communication, and continuous improvement. When defects are poorly documented, testers may report the same issue multiple times, developers may be unable to reproduce it, and managers may not have reliable information about release risk. A structured defect repository provides a central source of information throughout the defect lifecycle.

Each defect should be documented with a unique ID, title, description, affected module, environment, build version, preconditions, reproduction steps, expected result, actual result, severity, priority, status, owner, root cause, and resolution. Evidence such as logs or screenshots can be attached when useful. This information allows developers to reproduce the failure and testers to verify the correction.

A repository also supports analysis. Defects can be grouped by module, release, type, severity, root cause, and detection phase. For example, if a large number of defects are associated with the payment module, the team can investigate whether the cause is unclear requirements, interface problems, insufficient validation, or inadequate regression testing. Trend analysis can identify recurring categories and guide preventive action.

Documentation practices can be improved through standardized templates, mandatory fields, consistent classification, duplicate checks, regular defect triage, and root-cause analysis. The repository should also be linked to test cases and requirements when possible. This creates traceability from requirement to test, failure, correction, and verification. As a result, defect management becomes organized, measurable, and useful for preventing similar problems in future releases.

20. Redesigning Testing for Repeated Banking Transaction Failures

Answer

Repeated transaction failures in a banking system indicate that testing is not sufficiently systematic, risk-focused, or comprehensive. The testing approach should be redesigned around requirements, transaction risks, structured test design, automation, defect analysis, and regression testing. Testing should begin early in the SDLC and continue through integration, system, security, performance, and maintenance activities.

For transaction processing, test cases should cover valid transactions, insufficient balance, invalid account details, transaction limits, duplicate requests, network interruption, timeout, server failure, database failure, concurrent transactions, and recovery after partial completion. Boundary value analysis can test transaction limits, while decision tables can verify combinations of account status, balance, authorization, and transaction

type. State transition testing can verify states such as initiated, authorized, processed, failed, reversed, and completed.

Automation should be used for stable and repetitive regression scenarios, while exploratory testing can investigate unusual combinations and new risks. Every defect should be recorded in a repository and analyzed for root cause. If a defect occurs repeatedly, the related test case should be added or strengthened in the regression suite. Monitoring and test metrics such as defect density, failure rate, coverage, and escaped defects can be used to evaluate improvement.

For example, if money is deducted when a transaction fails because of a network interruption, the test team should reproduce the condition, verify transaction consistency, and ensure that recovery or reversal occurs correctly. Structured testing would then include this scenario in future regression cycles. This approach improves defect detection, reduces repeated failures, strengthens transaction reliability, and increases customer confidence in the banking system.



Code of Conduct:

I Yamini C G 192412054 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Yamini C G

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand